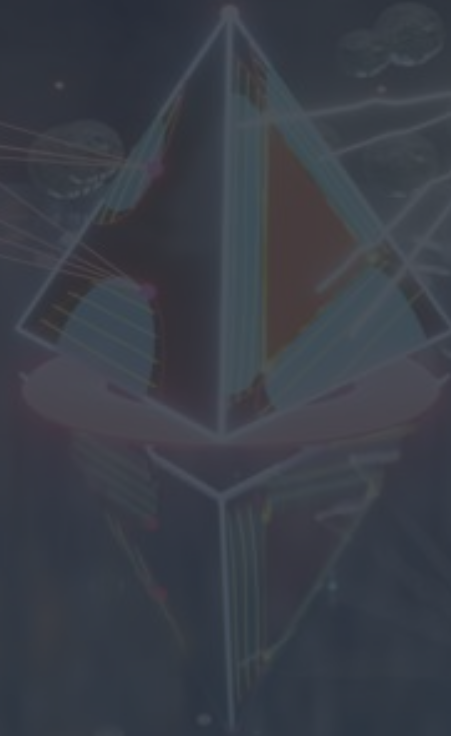
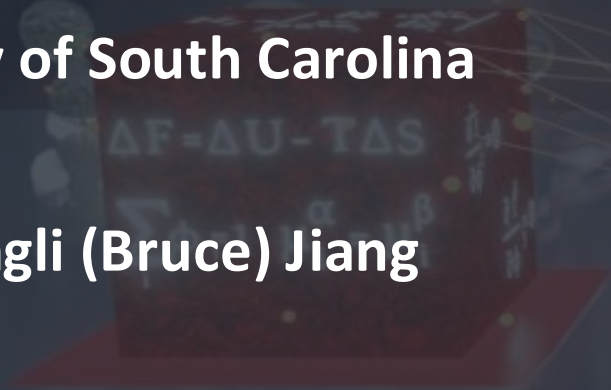


Regression: Linear Models

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



The Least-Squares Regression Problem

Machine learning Regression is often characterized as “fancy curve-fitting”; to understand the (un)fairness of that statement, we will first describe good ole regular curve-fitting.

Least-Squares Linear Regression

$$f(x) = \theta_0 + \theta_1 x$$

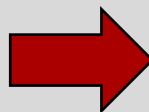
Given $\{(\mathbf{x}_i, y_i)\}$ produce “optimal” f , where $\hat{y} = f(\mathbf{x}, \boldsymbol{\theta})$ that minimizes some error metric (“loss”) $\mathcal{E}(\{y_i, \hat{y}_i\})$.

Error

$$e_k = \hat{y}_k - y_k$$

L₂ norm

$$\mathcal{E}_2 = \left(\sum_{i=1}^n |e_i|^2 \right)^{1/2}$$



Objective:

$$\begin{aligned} \min_{\boldsymbol{\theta}} \mathcal{E}(f) &= \min_{\boldsymbol{\theta}} \sum_{i=1}^n |e_i|^2 \\ &= \min_{\boldsymbol{\theta}} \sum_{i=1}^n (\underbrace{\theta_0 + \theta_1 x_i}_{\hat{y}_i} - y_i)^2 \end{aligned}$$

*Only the **L₂-based** objectives are least squares.*

Create a Model in Scikit-Learn

scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

🏠 > API Reference > sklearn.linear_model > LinearRegression

LinearRegression

Where to import it from

`class sklearn.linear_model.LinearRegression(*,
fit_intercept=True, copy_X=True, tol=1e-06, n_jobs=None,
positive=False)`

What is this * ?

[\[source\]](#)

Ordinary least squares Linear Regression.

LinearRegression fits a linear model with coefficients $w = (w_1, \dots, w_p)$ to minimize the residual sum of squares between the observed targets in the dataset, and the targets predicted by the linear approximation.

```
from sklearn.linear_model import LinearRegression  
  
model = LinearRegression()
```

Create the model

Parameters and
default values

The bare * means all parameters after it must be given by name.

LinearRegression(
fit_intercept=False)

LinearRegression(
False)



Understand Model Parameters

Parameters:

fit_intercept : *bool, default=True*

Whether to calculate the intercept for this model. If set to False, no intercept will be used in calculations (i.e. data is expected to be centered).

copy_X : *bool, default=True*

If True, X will be copied; else, it may be overwritten.

tol : *float, default=1e-6*

The precision of the solution (`coef_`) is determined by `tol` which specifies the convergence criterion of the underlying solver. `tol` is set as `atol` and `btol` of `scipy.sparse.linalg.lsqr` when fitting on sparse training data. `tol` is set as `cond` of `scipy.linalg.lstsq` when fitting on dense training data.

- We usually begin with the defaults.
- We change a parameter only when our data or modeling assumptions require it.

Fit, Inspect, and Predict

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])

#  $y = 1 * x_0 + 2 * x_1 + 3$ 
y = np.dot(X, np.array([1, 2])) + 3
```

```
model = LinearRegression()
model.fit(X, y)

print(model.coef_)
print(model.intercept_)
print(model.predict(np.array([[3, 5]])))
```

```
[1.  2.]
3.00000000000000018
[16.]
```

Read the Fitted Attributes

Attributes:

coef_ : *array of shape (n_features,) or (n_targets, n_features)*

Estimated coefficients for the linear regression problem. If multiple targets are passed during the fit (y 2D), this is a 2D array of shape (n_targets, n_features), while if only one target is passed, this is a 1D array of length n_features.

rank_ : *int*

Rank of matrix `X`. Only available when `X` is dense.

singular_ : *array of shape (min(X, y),)*

Singular values of `X`. Only available when `X` is dense.

intercept_ : *float or array of shape (n_targets,)*

Independent term in the linear model. Set to 0.0 if `fit_intercept = False`.

n_features_in_ : *int*

Number of features seen during [fit](#).

- A trailing underscore “xxx_” usually means information learned or determined during `fit()`.

Write Least Squares in Matrix Form

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Why 2n?

- θ represents any candidate parameters.
- $\hat{\theta}$ represents the fitted parameters that minimize the loss.

Convert the problem into matrix form

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & \cdots & x_{1p} \\ 1 & x_{21} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix}$$



$$\mathbf{e} = \mathbf{X}\theta - \mathbf{y}$$

$$J(\theta) = \frac{1}{2n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2$$

gradient = 0 is min?

Linear Means Linear in the Parameters

$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$ is still linear regression because each parameter appears linearly. **“Linear” means linear in the fitted coefficients.**

$$\phi_0(x) = 1, \quad \phi_1(x) = x, \quad \phi_2(x) = x^2$$

$$\hat{y} = \theta_0 \phi_0(x) + \theta_1 \phi_1(x) + \theta_2 \phi_2(x)$$

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ \vdots & \vdots & \vdots \\ 1 & x_n & x_n^2 \end{bmatrix} \quad \hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\theta}$$

Linear or nonlinear regression?

$$\hat{y} = \theta_0 + \theta_1 \sin(x) + \theta_2 \log(x)$$

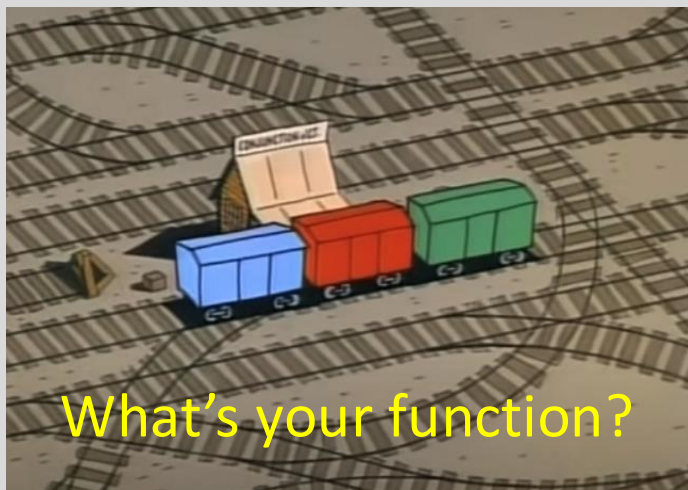
$$\hat{y} = \theta_0 + \exp(\theta_1 x)$$

$$\hat{y} = \frac{\theta_1 x}{\theta_2 + x}$$

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1 x_2$$

Optimization and Gradients in Machine Learning

computation of gradients and optimization methods are foundational to the success of machine learning, predominantly for training models and adapting



$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \mathbf{x} \mapsto f(\mathbf{x})$$

Partial derivatives are defined

$$\frac{\partial f}{\partial x_1} = \lim_{h \rightarrow 0} \frac{f(x_1 + h, x_2, \dots, x_n) - f(\mathbf{x})}{h}$$

$\vdots$

$$\frac{\partial f}{\partial x_n} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{n-1}, x_n + h) - f(\mathbf{x})}{h}$$

with a **gradient** expressed as

$$\nabla_{\mathbf{x}} f = \text{grad } f = \frac{df}{d\mathbf{x}} = \left[\frac{\partial f(\mathbf{x})}{\partial x_1} \quad \frac{\partial f(\mathbf{x})}{\partial x_2} \quad \dots \quad \frac{\partial f(\mathbf{x})}{\partial x_n} \right]^T \in \mathbb{R}^{n \times 1}$$

Practice Computing a Gradient

$$f(x_1, x_2) = x_1^2 + 3x_1x_2 + 2x_2^2$$

$$\nabla_{\mathbf{x}} f = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 + 3x_2 \\ 3x_1 + 4x_2 \end{bmatrix}$$

$$\nabla_{\mathbf{x}} f|_{\mathbf{x}=[1,2]^T} = \begin{bmatrix} 8 \\ 11 \end{bmatrix}$$

(Optional) Derive the Least-Squares Gradient

$$f(\boldsymbol{\theta}) = \frac{1}{2n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 \quad \longrightarrow \quad \nabla_{\boldsymbol{\theta}} f = \frac{1}{n} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

Steps:

$$\mathbf{X} \in \mathbb{R}^{n \times p}, \quad \boldsymbol{\theta} \in \mathbb{R}^{p \times 1}, \quad \mathbf{y} \in \mathbb{R}^{n \times 1}$$

$$r_i = \sum_{j=1}^p X_{ij} \theta_j - y_i \qquad f(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n r_i^2$$

$$\frac{\partial f}{\partial \theta_k} = \frac{1}{n} \sum_{i=1}^n X_{ik} r_i \qquad \nabla_{\boldsymbol{\theta}} f = \frac{1}{n} \mathbf{X}^T \mathbf{r}$$

Set the Gradient to Zero

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Define the optimum mathematically

At the least-squares optimum, the gradient is zero:

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 \quad \nabla_{\boldsymbol{\theta}} J = \frac{1}{n} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

$$\nabla_{\boldsymbol{\theta}} J = 0 \quad \Rightarrow \quad \mathbf{X}^\top \mathbf{X} \hat{\boldsymbol{\theta}} = \mathbf{X}^\top \mathbf{y} \quad \Rightarrow \quad \hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Why Solve Least Squares with SVD?

$$\hat{\theta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

We rarely compute this inverse directly.

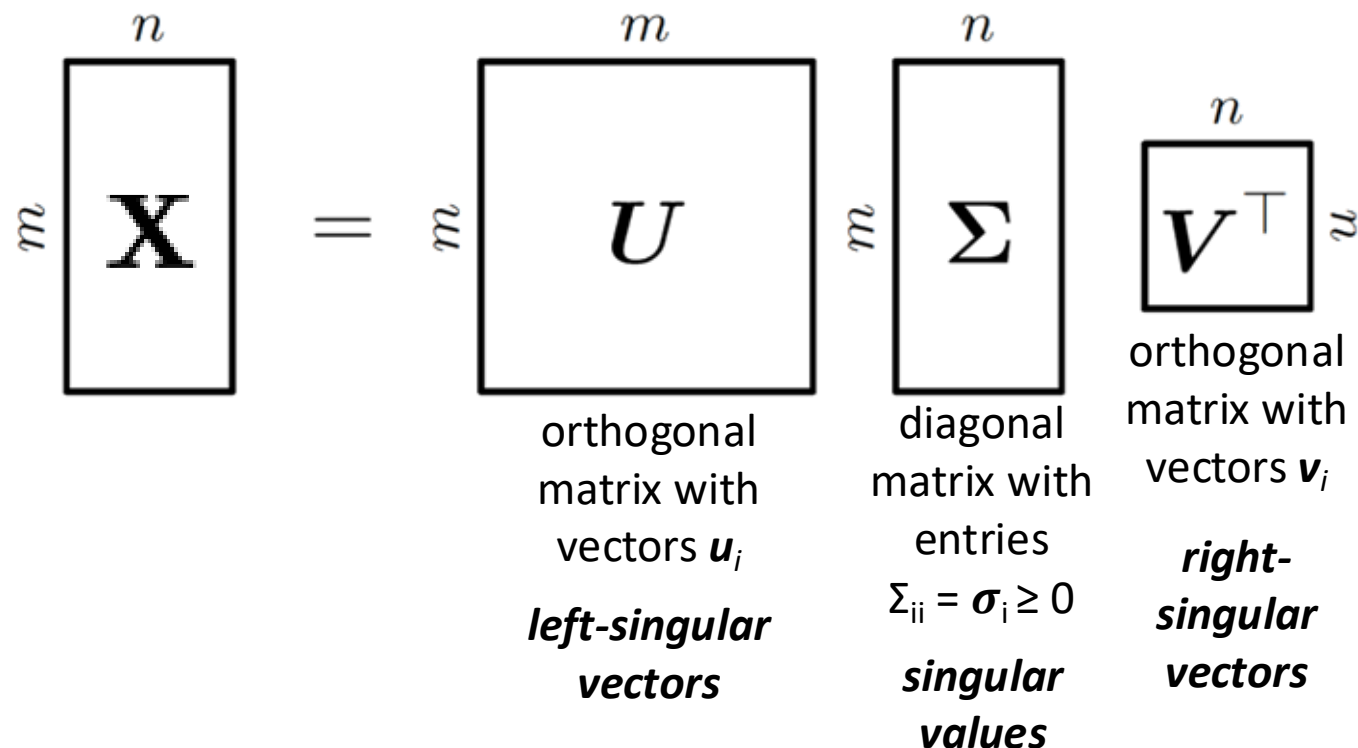
*Solve least squares without forming
or inverting $\mathbf{X}^\top \mathbf{X}$*

$$\mathbf{X} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$$

$$\hat{\mathbf{y}} = \mathbf{X} \theta$$

$$\hat{\mathbf{y}} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top \theta$$

Singular value decomposition
(**SVD**, the never-fail decomposition):



Construct SVD from Eigenvectors (Boring Math Again)

$$\mathbf{X} \in \mathbb{R}^{n \times p}$$

1. For a symmetric matrix

$$\mathbf{B} = \mathbf{X}\mathbf{X}^\top \in \mathbb{R}^{n \times n}$$

2. Compute its eigenvectors and eigenvalues

$$\mathbf{B}\mathbf{u}_i = \lambda_i \mathbf{u}_i \quad \det(\mathbf{B} - \lambda \mathbf{I}) = 0 \quad \mathbf{U} = [\mathbf{u}_1 \quad \mathbf{u}_2 \quad \cdots]$$

3. Obtain the singular values

$$\sigma_i = \sqrt{\lambda_i} \quad \text{largest to smallest} \quad \mathbf{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_r)$$

4. Compute the right singular vectors

$$\mathbf{X}^\top \mathbf{u}_i = \sigma_i \mathbf{v}_i \quad \mathbf{v}_i = \frac{\mathbf{X}^\top \mathbf{u}_i}{\sigma_i} \quad \mathbf{V} = [\mathbf{v}_1 \quad \mathbf{v}_2 \quad \cdots]$$

SVD Example

$$y = \theta_0 + \theta_1 x \quad (x_1, y_1) = (-1, 1), \quad (x_2, y_2) = (0, 3), \quad (x_3, y_3) = (1, 5)$$

Step 0: represent this in a matrix form

$$\mathbf{X} = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 3 \\ 5 \end{bmatrix}$$

Step 1: form a symmetric matrix

$$\mathbf{B} = \mathbf{X}\mathbf{X}^T = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Step 2: Compute its eigenvectors and eigenvalues

$$\begin{aligned} \det(\mathbf{B} - \lambda\mathbf{I}) &= 0 & \lambda_1 &= 3 \\ -\lambda(3 - \lambda)(2 - \lambda) &= 0 & \lambda_2 &= 2 \\ & & \lambda_3 &= 0 \end{aligned}$$

$$\mathbf{u}_1 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}, \quad \mathbf{u}_3 = \frac{1}{\sqrt{6}} \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

$$\mathbf{U} = \begin{bmatrix} \frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & 0 & \frac{2}{\sqrt{6}} \\ \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}} \end{bmatrix}$$

SVD Example

$$y = \theta_0 + \theta_1 x \quad (x_1, y_1) = (-1, 1), \quad (x_2, y_2) = (0, 3), \quad (x_3, y_3) = (1, 5)$$

Step 3: obtain the singular values

$$\sigma_1 = \sqrt{3}, \quad \sigma_2 = \sqrt{2}, \quad \sigma_3 = 0$$

Because $\mathbf{X}$ is 3x2

$$\mathbf{\Sigma} = \begin{bmatrix} \sqrt{3} & 0 \\ 0 & \sqrt{2} \\ 0 & 0 \end{bmatrix}$$

Step 4: Compute the right singular vectors

$$\mathbf{v}_1 = \frac{\mathbf{X}^T \mathbf{u}_1}{\sigma_1} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \mathbf{v}_2 = \frac{\mathbf{X}^T \mathbf{u}_2}{\sigma_2} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \mathbf{V} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Final Step

$$\mathbf{X} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

$$\hat{\boldsymbol{\theta}} = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^T \mathbf{y} = \begin{bmatrix} 3 \\ 2 \end{bmatrix} \quad \boxed{\hat{y} = 3 + 2x}$$

$$\mathbf{\Sigma}^+ = \begin{bmatrix} \frac{1}{\sqrt{3}} & 0 & 0 \\ 0 & \frac{1}{\sqrt{2}} & 0 \end{bmatrix} \quad \sigma_i^+ = \begin{cases} \frac{1}{\sigma_i}, & \sigma_i > 0, \\ 0, & \sigma_i = 0. \end{cases}$$

+ is: Moore–Penrose pseudoinverse

How Scikit-Learn Solves the Problem

```
from sklearn.linear_model import LinearRegression  
  
model = LinearRegression()  
model.fit(X, y)
```



`scipy.linalg.lstsq(X, y)`



LAPACK GELSD



divide-and-conquer SVD
(same math, faster algorithm)

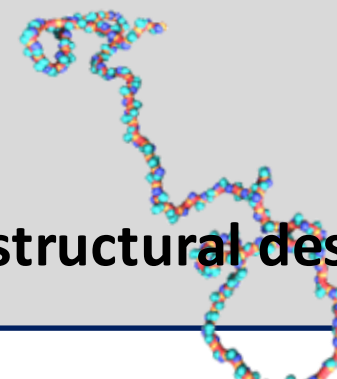
$$\hat{\theta} = \mathbf{V}\Sigma^+\mathbf{U}^T\mathbf{y}$$

Predict Polymer Glass-Transition Temperature

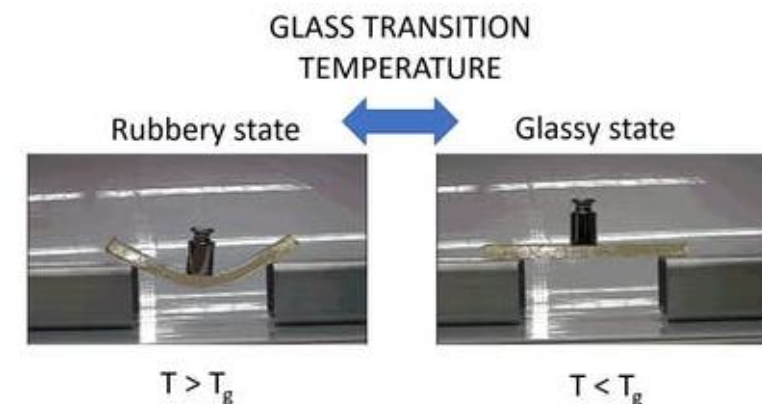
- A polymer is a large molecule made of many repeating molecular units.
- Polymer structure strongly affects material properties.

The **glass-transition temperature, T_g** , is the temperature at which an amorphous polymer changes from a hard, glassy state to a softer, more flexible state.

In this synthetic dataset, T_g is predicted from three structural descriptors.



- `aromatic_fraction` measures the fraction of rigid aromatic groups
- `rotatable_bond_fraction` measures molecular flexibility
- `hbond_sites_per_repeat` counts hydrogen-bonding sites per repeat unit
- `tg_c` is the glass-transition temperature in degrees Celsius



Load and Inspect the Polymer Data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

data = pd.read_csv("polymer_tg_data.csv")

data.head()
```

	polymer_id	aromatic_fraction	rotatable_bond_fraction	hbond_sites_per_repeat	tg_c
0	P01	0.08	0.23	1	19.8
1	P02	0.25	0.52	0	9.8
2	P03	0.42	0.81	3	61.9
3	P04	0.59	0.34	2	141.1
4	P05	0.76	0.63	1	113.2

Explore Descriptor–Property Relationships

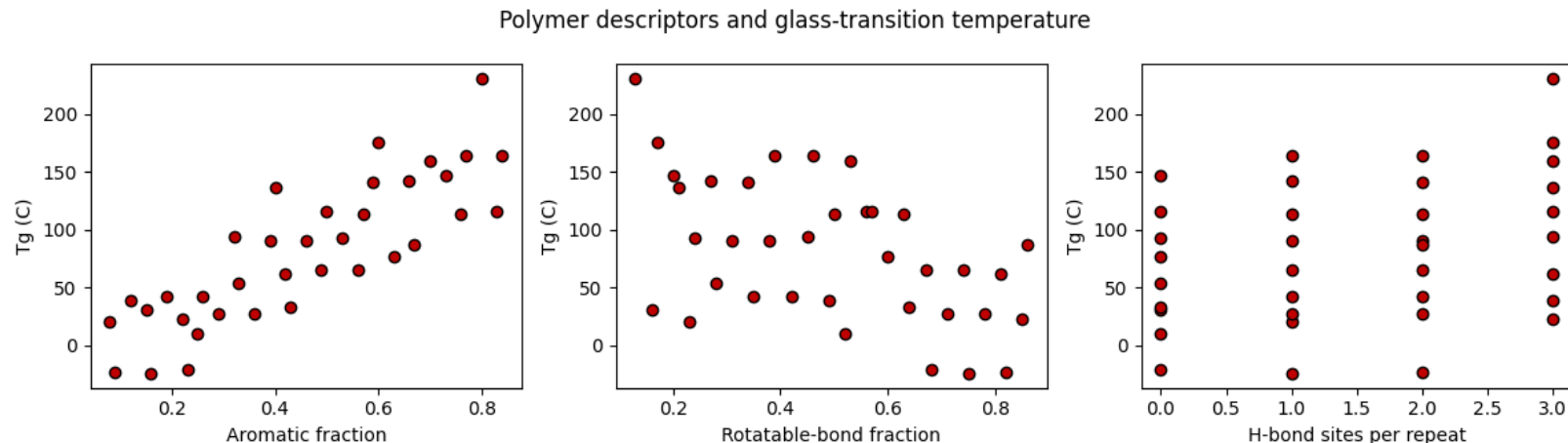
```
features = ["aromatic_fraction", "rotatable_bond_fraction", "hbond_sites_per_repeat"]

X = data[features].to_numpy()
y = data["tg_c"].to_numpy()

labels = ["Aromatic fraction", "Rotatable-bond fraction", "H-bond sites per repeat"]
fig, axs = plt.subplots(1, 3, figsize=(12, 3.5))

for ax, feature, label in zip(axs, features, labels):
    ax.scatter(data[feature], y, color="#C00000", edgecolor="black")
    ax.set_xlabel(label)
    ax.set_ylabel("Tg (C)")

fig.suptitle("Polymer descriptors and glass-transition temperature")
fig.tight_layout()
plt.show()
```



Fit the Model with Scikit-Learn

```
model = LinearRegression()  
model.fit(X, y)  
y_pred = model.predict(X)
```

```
print(model.intercept_)  
print(model.coef_)
```

```
15.895083811721122  
[ 207.97197839 -122.33678518   20.38068656]
```

```
rmse = np.sqrt(np.mean((y - y_pred) ** 2))  
print(rmse)
```

```
3.6177136730503645
```

```
lower = min(y.min(), y_pred.min())  
upper = max(y.max(), y_pred.max())
```

```
fig, ax = plt.subplots(figsize=(5, 5))
```

```
ax.scatter(y, y_pred, color="#C00000", edgecolor="black")  
ax.plot([lower, upper], [lower, upper], "--", color="#1B2A49")
```

```
ax.set_xlabel("True Tg (C)")  
ax.set_ylabel("Predicted Tg (C)")  
ax.set_title("Parity plot")  
ax.set_xlim(lower, upper)  
ax.set_ylim(lower, upper)
```

```
ax.set_aspect("equal", adjustable="box")
```

```
fig.tight_layout()  
fig.show()
```

Evaluate the Training Fit

```
lower = min(y.min(), y_pred.min())
upper = max(y.max(), y_pred.max())

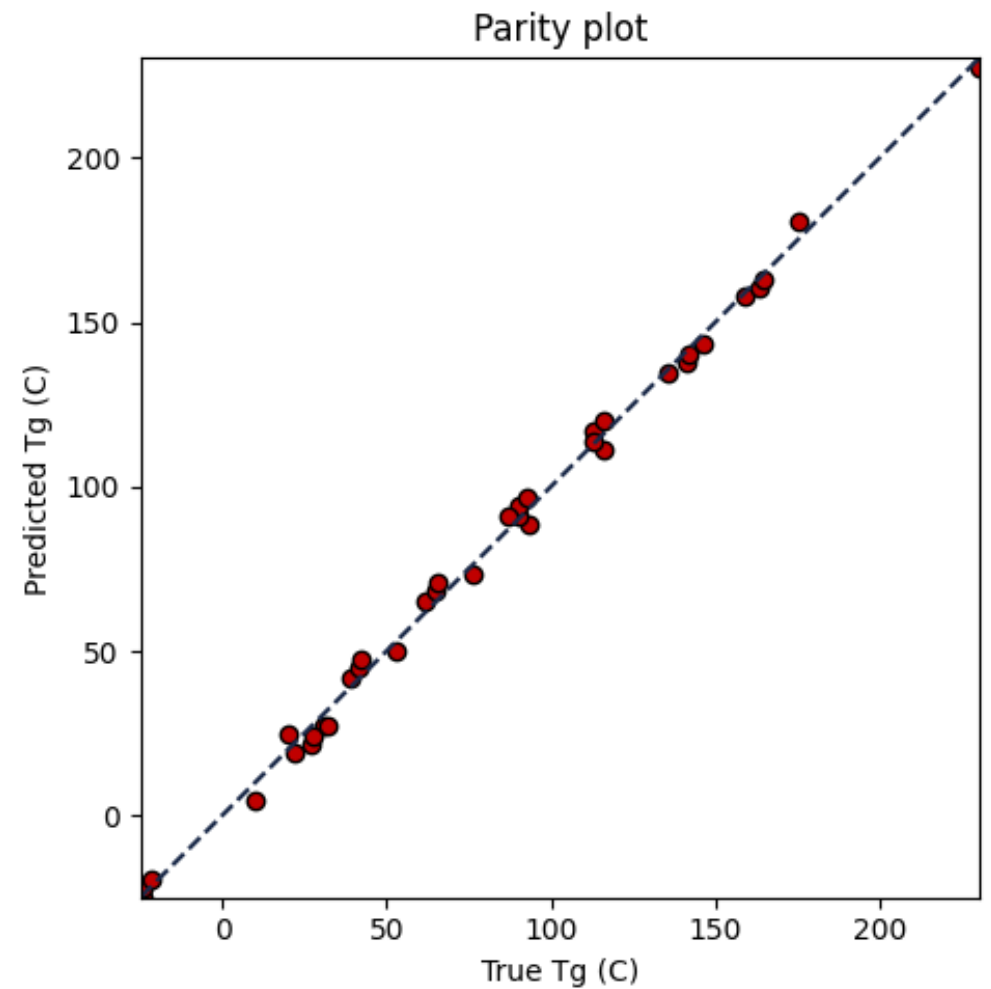
fig, ax = plt.subplots(figsize=(5, 5))

ax.scatter(y, y_pred, color="#C00000", edgecolor="black")
ax.plot([lower, upper], [lower, upper], "--", color="#1B2A49")

ax.set_xlabel("True Tg (C)")
ax.set_ylabel("Predicted Tg (C)")
ax.set_title("Parity plot")
ax.set_xlim(lower, upper)
ax.set_ylim(lower, upper)

ax.set_aspect("equal", adjustable="box")

fig.tight_layout()
fig.show()
```



Reproduce the Fit with SVD

$$\hat{\theta} = V\Sigma^+U^T y$$

```
A = np.column_stack((np.ones(X.shape[0]), X))

U, s, Vt = np.linalg.svd(A, full_matrices=False)

Sigma_inv = np.diag(1 / s)

theta = Vt.T @ Sigma_inv @ U.T @ y

y_pred = A @ theta

print(A.shape)
print(A[:4, :])
print(U.shape, s.shape, Vt.shape)
```

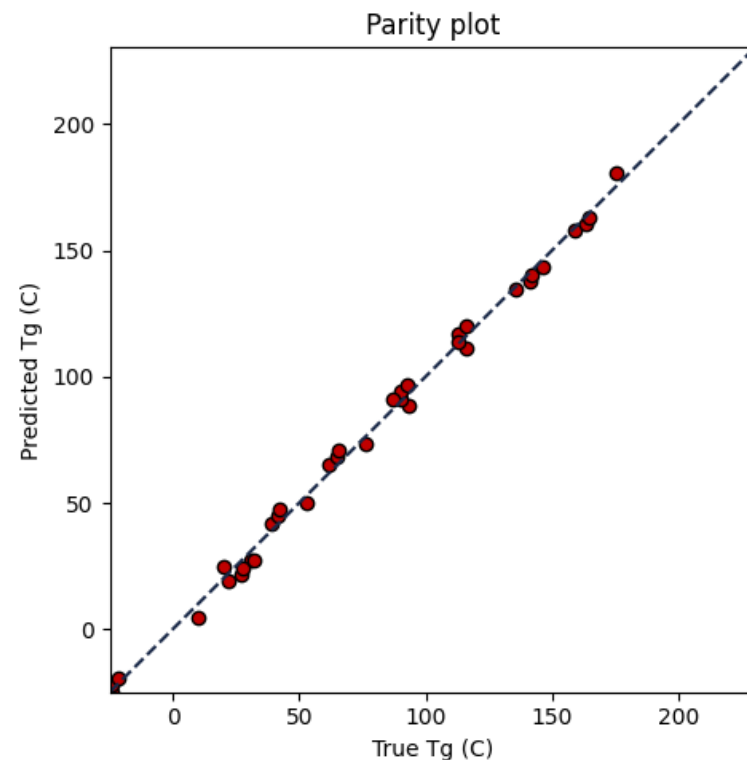
```
(36, 4)
[[1.    0.08 0.23 1.   ]
 [1.    0.25 0.52 0.   ]
 [1.    0.42 0.81 3.   ]
 [1.    0.59 0.34 2.   ]]
(36, 4) (4,) (4, 4)
```

```
print(theta)
```

```
[ 15.89508381  207.97197839 -122.33678518  20.38068656]
```

```
rmse = np.sqrt(np.mean((y - y_pred) ** 2))
print(rmse)
```

```
3.6177136730503654
```



Interactive Python Time

05_linear_regression.ipynb